SHORTEST PATH FINDER DEMONSTRATION

DAA Mini Project • Programming Language: Python 3

Project Type	Design and Analysis of Algorithms Mini Project
Problem Domain	Graph algorithms / route planning
Programming Language	Python 3.10+
Algorithms Demonstrated	Dijkstra, Bellman–Ford, Floyd–Warshall
Repository	GitHub repository containing source code, README, tests, screenshots and this report

Submission Contents

- **GitHub Repository Link:** The repository URL submitted with the project.
- **Source Code:** Python implementation with separate algorithm, utility and test modules.
- **Project Report:** This PDF containing the problem, objectives, algorithm design, flowchart, technologies, complexity analysis, test cases, output screenshots and conclusion.

Project Overview

The Shortest Path Problem is modeled as a weighted directed graph in which locations are vertices and travel costs (distance or time) are edge weights. The project demonstrates three solution strategies: a greedy single-source algorithm, a dynamic-programming single-source algorithm that supports negative weights, and a dynamic-programming all-pairs algorithm. Their capabilities and computational behavior are compared to illustrate important DAA trade-offs.

Single-source methods	Dijkstra (greedy) and Bellman–Ford (dynamic programming)
All-pairs method	Floyd–Warshall (dynamic programming)
Validation	Automated unit tests and matrix validation
Benchmarking	Runtime measurement using time.perf_counter()

1. Introduction

The shortest path problem is a classic graph optimization problem and an important example for studying algorithm design. Given a network of locations and the cost of travelling along each road, the objective is to find the minimum-cost route from a source to a destination (or between all pairs of locations). It is used in GPS navigation, network routing and logistics. This project uses it to demonstrate how different algorithmic techniques affect capability, running time and scalability.

2. Problem Statement

Given a weighted directed graph G = (V, E) with edge weights w(u, v) and a source vertex s, find the minimum-cost path from s to every other vertex (single-source), and optionally between every pair of vertices (all-pairs). The project must also detect unreachable vertices and negative-weight cycles, and compare the efficiency of the algorithms.

3. Objectives

- Model the road network as a weighted graph using an adjacency matrix.
- Implement a greedy algorithm and two Dynamic Programming algorithms.
- Establish correctness of the implemented approaches.
- Analyze time and space complexity.
- Measure practical execution time and compare algorithm behavior.

4. Algorithm Design

Dijkstra	Keep a min-heap of (distance, vertex). Repeatedly extract the closest unfinalized vertex and relax its outgoing edges. Greedy choice: the closest vertex's distance is final. Requires non-negative weights.
Bellman–Ford	Initialize dist[s] = 0. Relax every edge V−1 times; if an edge can still be relaxed in the V-th pass, a negative cycle exists. Handles negative weights.
Floyd–Warshall	Define dist[i][j] after considering intermediate vertices 0..k. Update dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]) for every k. Gives all-pairs shortest paths.

5. Correctness

Dijkstra is correct because, with non-negative weights, the vertex with the smallest tentative distance cannot be improved by any later path (greedy-choice property). Bellman–Ford is correct because after i passes all shortest paths using at most i edges are found, and a shortest simple path has at most V−1 edges. Floyd–Warshall is correct by optimal substructure: the best path through intermediate vertices {0..k} either avoids k or passes through k exactly once. All three were cross-checked against each other on random graphs by the automated tests.

6. Flowchart

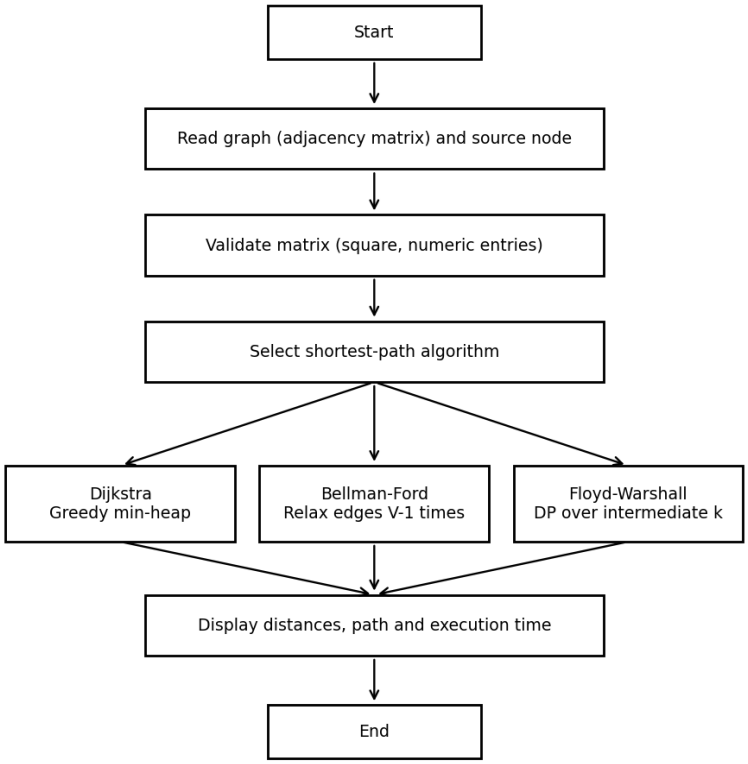


Figure 1. Overall shortest-path processing flow

7. Technologies / Programming Language

Python 3.10+ and standard-library modules are used (heapq for the priority queue). Automated tests use unittest and runtime measurement uses time.perf_counter(). No external runtime dependencies are required.

8. Complexity Analysis (Time & Space)

Algorithm	Time	Space	Negative edges?	Type
Dijkstra (binary heap)	O((V + E) log V)	O(V)	No	Single-source
Bellman–Ford	O(V × E)	O(V)	Yes (detects cycles)	Single-source
Floyd–Warshall	$O(V^3)$	$O(V^2)$	Yes (detects cycles)	All-pairs

Dijkstra is the fastest because its greedy choice avoids re-scanning edges, but it cannot handle negative weights. Bellman–Ford pays an extra factor of V to support negative weights and cycle detection. Floyd–Warshall solves every source at once in cubic time, which is efficient only for dense or small graphs.

9. Test Cases and Output Screenshots

Test	Condition	Expected Result
TC01	5-node sample graph from A	Distances 0, 10, 50, 30, 60 from all algorithms
TC02	Graph with a negative edge	Bellman–Ford / Floyd correct; Dijkstra rejects input
TC03	Graph with a negative cycle	NegativeCycleError raised
TC04	Single node and unreachable node	Distance 0; unreachable = INF, empty path
TC05	Random graphs (5, 10, 25 nodes)	All three algorithms agree for every source
TC06	Path reconstruction	Path A → D → C → E
TC07	Invalid matrix	Validation error

Sample input: 5-node distance matrix (INF = no direct road)

	A	B	C	D	E
A	0	10	INF	30	100
B	INF	0	50	INF	INF
C	INF	INF	0	INF	10
D	INF	INF	20	0	60
E	INF	INF	INF	INF	0

The shortest distance from A to E is 60 (path A → D → C → E); the direct edge A → E costs 100.

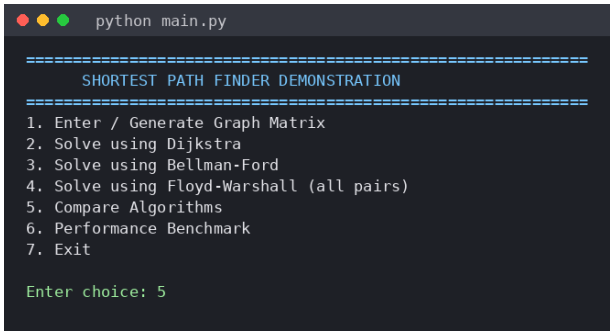


Figure 2. Main menu

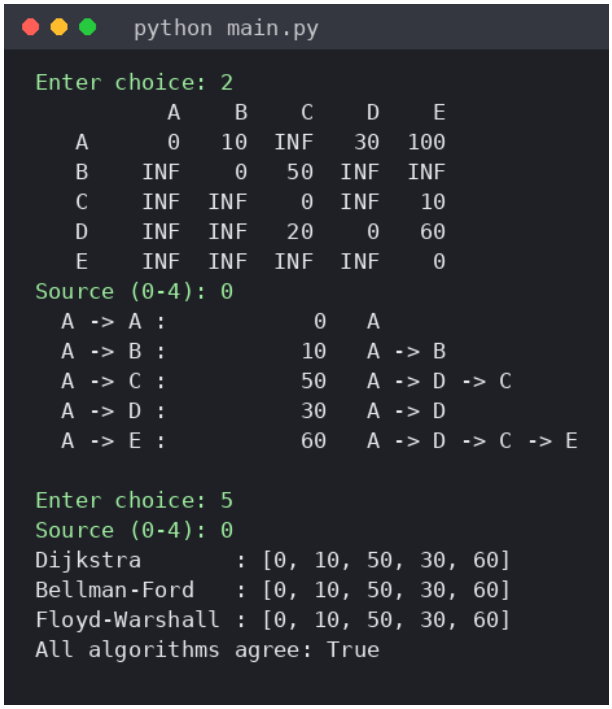


Figure 3. Algorithm result

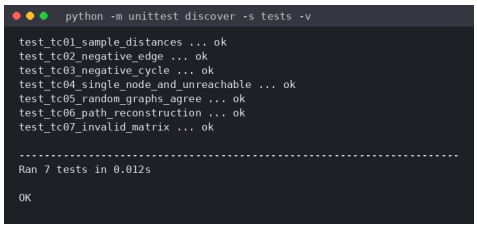


Figure 4. All 7 automated unit tests pass

10. Performance Comparison

Nodes	Algorithm	Sum of distances	Median Time (sec)
10	Dijkstra	190	0.00001145
10	Bellman-Ford	190	0.00002033
10	Floyd-Warshall	190	0.00007626
20	Dijkstra	789	0.00003380
20	Bellman-Ford	789	0.00006173
20	Floyd-Warshall	789	0.00051087
40	Dijkstra	758	0.00013152
40	Bellman-Ford	758	0.00028474
40	Floyd-Warshall	758	0.00397835
80	Dijkstra	882	0.00047437
80	Bellman-Ford	882	0.00148006
80	Floyd-Warshall	882	0.04923588

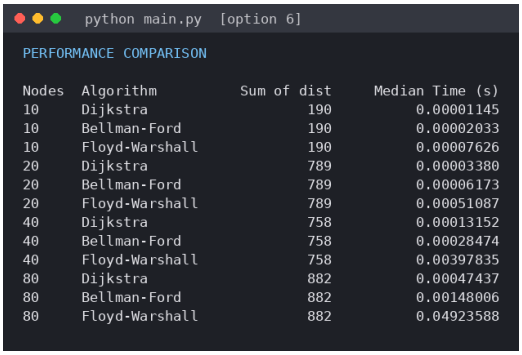


Figure 5. Performance comparison generated by the project

Measured runtime depends on hardware and runtime conditions; asymptotic complexity is the primary theoretical comparison. All three algorithms return identical distances (same sum), while Floyd–Warshall grows fastest because it computes all pairs.

11. Conclusion

The project connects graph modeling, greedy design, Dynamic Programming, correctness, complexity analysis, testing and practical benchmarking. The three methods make the trade-off between speed and capability visible: Dijkstra is fastest but needs non-negative weights, Bellman–Ford supports negative weights, and Floyd–Warshall answers every pair at higher cost.

12. Future Scope

A* search, bidirectional Dijkstra, Johnson's algorithm, graphical visualization, benchmark plots and real map data can be added.

13. Repository / Execution

https://github.com/YOUR-USERNAME/shortest-path-finder-daa.git